

Solving the Full N-Dimensional Rubik's Supercube on the GPU with a Seeded Genetic Algorithm

Robert Świta, Mateusz Kosikowski, Zbigniew Suszyński

Faculty of Electronics and Computer Science, Koszalin University of Technology

Department of Multimedia Systems and Artificial Intelligence

Abstract

We present a method for solving the full N-dimensional Rubik's supercube — a Size^N hypercube of cubies, each carrying its own orientation, including the internal ones — with a genetic algorithm executed entirely on the GPU as OpenGL compute shaders. Building on a matrix formalization in which cubie orientation is an N-dimensional affine transform decomposed into Euler angles by QR factorization with Givens rotations, we contribute the following. First, a moves-to-solve fitness that augments the L1 orientation distance with the number of active axes of each cubie, together with an endgame reweighting that removes the mono-twist trap dominant for $N > 3$. Second, GPU-native genetic operators: a reverse-complement (commutator) crossover and a macro-mutation that re-injects per-cubie solving sequences every generation. Third, a diversity-preserving unique selection folded into the bitonic sort. Fourth, a reject-free greedy generator of per-cubie solving sequences that covers every move-reachable decomposition tree. Fifth, a near-free dead-move penalty — a gene prefix that leaves the active cluster unchanged is rejected by comparing packed orientations rather than fitness — which, paired with a stall-driven perturbation that reapplies the least-worsening real move, escapes the local minima that produce the heavy runtime tail. The fitness is computed by a single source of truth on the device. The solver orders clusters from the centre outward and reaches configurations such as the 6x6x6 cube in minutes, and the full four-dimensional 3^4 cube in about two minutes, without any precomputed macro-move database.

Keywords: N-dimensional Rubik's cube, genetic algorithm, GPU compute shaders, Givens rotations, group theory, combinatorial optimization, iterated local search.

1. Introduction

The Rubik's cube, invented in 1974, is one of the best-known combinatorial puzzles; the classic 3x3x3 admits about 4.3×10^{19} configurations. In the N-dimensional generalization the cube becomes a hypercube of Size^N cubies, and rotations no longer occur about axes but in two-dimensional planes spanned by pairs of coordinate axes. The number of such planes is $N(N-1)/2$ — three for 3D, six for 4D, ten for 5D. In previous work the authors solved the full 3D supercube (accounting for the orientation of every cubie, including the interior) with a genetic algorithm that arranges the cube in stages, from the centre outward, restricting moves to an active cluster [1].

This paper generalizes that approach to arbitrary $N \geq 3$ and moves the entire search onto the graphics processor. The evaluation of a population is embarrassingly parallel: each of POPULATION individuals independently applies GENES moves to CUBIES cubies. A whole generation — initialization, evaluation, sorting and reproduction — runs without returning to the host. The contributions are:

- a formalization of the N-dimensional move set and cubie orientation as packed signed-permutation matrices, with a fitness based on the true number of moves needed to solve each cubie (Section 3.1);
- an endgame reweighting that prevents the search from stalling on a single badly-oriented cubie (a mono-twist), a failure mode prevalent for $N > 3$ (Section 3.2);
- GPU-native genetic operators: a commutator crossover, a re-injecting macro-mutation, and a unique selection embedded in the bitonic sort (Sections 3.3-3.5);
- a reject-free greedy procedure that generates diverse per-cubie solving sequences (seeds) covering every move-reachable decomposition tree (Section 3.6);
- a dead-move penalty and a stall-driven perturbation that together escape local minima at every stage of cluster completion (Sections 3.7-3.8).

2. Formal Model of the N-Dimensional Cube

2.1 Affine representation of cubies

An affine transformation in N-dimensional space is a pair $T = (M, p)$ of an $N \times N$ matrix and a translation vector, acting as $T(x) = Mx + p$. Composition of two affine transforms is again affine, with $M' = M_2 M_1$ and $p' = M_2 p_1 + p_2$. Each cubie is a rigid body described by such a transform. A solved cubie is $T = (S, p)$ with S a diagonal scaling and p its home position. A cube move applies an orthogonal rotation $R = (Q, 0)$ to every cubie of the turned layer, and successive rotations compose into the cubie's total transform $T' = (Q'S, p')$, where Q' is the accumulated orientation (state) and $p' = Q'p$ the current position. Position and orientation are therefore not stored independently: the current position follows from the orientation applied to the home position, so orientation is the only mutable state. A cubie is solved iff its orientation is the identity. Cubies are held in a one-dimensional array indexed by the linear index of their home coordinates; a transformation never changes a cubie's slot, only its transform.

2.2 Orientation as Euler angles by QR decomposition

A rotation in the plane spanned by axes (m, n) is a Givens rotation — the identity except for a 2×2 block $\begin{bmatrix} \cos & -\sin \\ \sin & \cos \end{bmatrix}$ at the intersections of rows and columns m, n . Any orientation Q' factorizes into a product of such rotations, whose angles — the Euler angles — are the multiples of 90 degrees needed in successive planes. Because the cube's moves are exactly 90-degree rotations, the state is captured by two bits per plane and the whole orientation by a single integer.

The factorization is obtained by QR decomposition using Givens rotations, which is numerically stable and cheap. Treating the orientation matrix M as input, the algorithm performs $P = N(N-1)/2$ iterations, each choosing a Givens angle that zeroes one sub-diagonal element, until M is reduced to a diagonal (identity up to sign). The recorded angles define the orientation; reversing them yields a sequence that solves the cubie. The order in which planes are eliminated is a free choice — a strategy — and different orders give different but equally valid factorizations. Since at most $N-1$ of the P planes carry a non-zero angle, the number of moves required to reach any orientation is bounded by $\text{RotationCount} \leq N-1$. The unique orientations of a cubie (orthogonal, determinant +1) number $N! \cdot 2^{N-1}$ — 24 for 3D, 192 for 4D, 1920 for 5D — which also bounds the number of cubies in one cluster.

2.3 Universal move encoding

An N-dimensional move is defined by four indices: the axis $\text{Axis} < N$ perpendicular to the turned layer, the layer $\text{Slice} < \text{Size}$ along that axis, the rotation plane $\text{Plane} < N(N-1)/2$, and the angle $\text{Angle} < 3$ (90, 180 or 270 degrees), packed into a single integer by linear indexing, $\text{Code} = ((\text{Axis} \cdot \text{Size} + \text{Slice}) \cdot P + \text{Plane}) \cdot 3 + \text{Angle}$, with the constraint that the move axis must differ from both axes of the rotation plane. The total number of legal moves is $N \cdot \text{Size} \cdot (N-1) \cdot 3$, so not every code in the range is valid — and, notably, no code denotes the identity: every gene is a real turn. This density is used by the dead-move penalty of Section 3.7.

2.4 Clusters

A cluster is the orbit of a cubie under the group of moves that affect it; all cubies sharing a cluster index (the linear index of the sorted magnitudes of their coordinates) belong to one cluster. The solver picks the unsolved cubie nearest the cube centre in the max metric as the active cubie, forms its cluster, restricts the search to the moves affecting that cluster (the free moves), solves it, and repeats outward until the whole cube is arranged. Sequences that finish a cluster can be remembered as reusable macro-moves; the method here needs no such database.

3. Genetic Algorithm

A chromosome is a fixed-length sequence of GENES encoded moves, restricted to the free moves of the active cluster, so every individual is a candidate manoeuvre acting only where allowed. The algorithm evolves POPULATION individuals for GENERATIONS generations per invocation, invoked once per cluster attempt.

3.1 Fitness by number of moves to solve

Evaluation applies the moves one by one and keeps the best prefix: the number of leading moves that yields the lowest error, with that error. The error separates two groups. The solved group holds cubies from finished clusters that must not be disturbed; each violated one adds a whole unit. The active group holds the current cluster; for an active cubie we define a per-cubie penalty, zero iff solved and otherwise dominated by how many moves it still needs:

$$\text{cubieState}(M) = m \cdot (\text{MAX_CUBIE_L1} + 1) + L_1(M),$$

where m is the number of active axes (rows of the packed matrix differing from the identity) and L_1 is the Manhattan distance of the packed orientation from the identity. A cubie with active-axis count m needs about $m-1$ moves to

solve (a spanning tree of Givens rotations over its active axes), so m is a direct moves-to-solve measure; weighting it above the entire L1 range makes the search minimise the number of moves first and refine the raw distance only within that — a strictly monotone gradient in move space that L1 alone does not provide. The cluster error is the normalized sum $(\text{Sum_active}(\text{maxClusterState} + \text{cubieState})) / \text{max_fA} + \text{Sum_solved} \cdot 1$, with $\text{maxClusterState} = \text{MAX_CUBIE_STATE} \cdot |A|$, $\text{max_fA} = \text{maxClusterState} \cdot (|A| + 1)$, so the active term stays strictly below one and the integer count of disturbed solved cubies always dominates it.

3.2 Endgame reweighting against the mono-twist

Reducing the count of unsolved cubies guides well early but misleads near the end: a configuration with very few unsolved cubies is not necessarily easier, and in 4D and higher a single cubie can remain twisted in isolation — a mono-twist — which is very hard to escape (unlike 3D, where the orientation-sum invariant forbids a lone twist, the non-abelian 4D orientation group admits it). We therefore switch the objective once the number of unsolved active cubies falls to N or fewer: the active error is multiplied by $N/\text{scrambled}$, where scrambled is that count. The count contribution becomes constant across the endgame, so the search stops chasing fewer cubies and minimises the amplified orientation magnitude; a lone hard cubie receives the maximum penalty (a factor of N). The switch is essential for $N > 3$.

3.3 Selection with uniqueness embedded in the sort

The population is sorted by fitness with a bitonic network, the fittest at index 0; parents are drawn by rank from the top WINNERS fraction. A converging population quickly fills with equal-fitness clones, collapsing the parent pool. Rather than a separate deduplication pass, a unique selection is folded into the sort: whenever the compare-and-exchange finds two equal-fitness individuals, one is demoted by a penalty equal to the maximum possible fitness (CUBIES). Distinct-fitness individuals rise while duplicates sink out of the winner pool. Because the penalty only increases fitness, the global minimum is never penalised and index 0 remains a true best; the penalty is transient, overwritten by the next evaluation. This single change was decisive in practice.

3.4 Crossover

Reproduction uses a reverse-complement crossover that builds commutators — the group-theoretic tool for permuting a few pieces without disturbing the rest. A child inherits a prefix from one parent and a middle block from the other; the second half of the chromosome is then the first half reversed and inverted (each move's angle negated). The result has the structure of a conjugate $A \cdot B \cdot A^{-1}$, from which the population assembles the 3-cycle and 2-orientation commutators that finish clusters.

3.5 Macro-mutation

The mutation operator is not a random gene change — which only degrades individuals — but a macro-mutation that overwrites the leading genes of a child with a fresh, complete sequence solving one active-cluster cubie, then conjugates a block of it. Applied to a fraction MUTATION of the children every generation, it continually re-injects cubie-solvers, which crossover recombines into multi-cubie solutions. Without this re-injection the seed material from initialization is diluted away within a few generations.

3.6 Seed generation by greedy shortest-solve

Both the initial seeding and the macro-mutation draw on a supply of per-cubie solving sequences, generated by a greedy shortest-solve of a random active cubie: at each step pick, uniformly at random, a cube move (a plane and a 90/180/270-degree angle) that lowers the cubie's RotationCount by one, record it, apply it, until the orientation is the identity. Because RotationCount is the graph distance to the solved state, a progress move always exists on some shortest path, so the procedure never gets stuck and needs no validation. Each move is realized by choosing any axis not in its rotation plane and reading the cubie's current layer along that axis; the plane rotation is applied regardless of the chosen axis, so the target's trajectory is axis-independent while the axis varies which other cubies are collaterally disturbed — an orthogonal, cheap source of diversity.

A combinatorial analysis over all reachable orientations confirms that this greedy generator covers every decomposition tree reachable by cube moves. For a fully-scrambled 4D cubie, the shortest solutions use $N-1 = 3$ rotation planes; counting distinct plane-sets yields 12 spanning trees, while counting distinct ordered sequences yields 16 (the extra four are non-commuting reorderings of the same plane-set). The RotationCount-driven greedy reaches all 16 ordered decompositions (12 sets), whereas the lighter variant below covers 12 of 16 (10 of 12 sets). We note that RotationCount is an order-dependent QR count, not a geometric distance, which is why a move touching an already-solved axis can still lower it; the greedy nonetheless remains reject-free over all reachable states up to 5D.

```

while (RotationCount(cubie) > 0):
    cand = [ (plane, angle) for all planes, angles in {90,180,270}
              if RotationCountAfter(cubie, plane, angle) == RotationCount - 1 ]
    (plane, angle) = random choice from cand # every choice is on a shortest path
    axis = any axis not in plane # collateral-diversity dimension
    slice = cubie.coordinate(axis) # layer currently holding the cubie
    emit Encode(axis, slice, plane, angle); apply the move to the cubie

```

The host draws distinct decompositions until the pool saturates (it stops after a run with no new tree; for $N = 3$ the distinct trees are often three or fewer, so the loop ends quickly instead of spinning). A lighter variant replaces `RotationCount` with the number of non-positive diagonal entries of the orientation matrix — read directly, no decomposition — likewise never stuck and covering 10 of the 12 trees.

3.7 Dead-move penalty

In a local minimum the fitness rewards doing nothing — the baseline score is the smallest attainable — so the population converges to individuals whose net move is the identity, and the best returned by a generation does nothing. To prevent this, the evaluator penalizes a gene prefix that leaves the active cluster unchanged, detected almost for free by comparing each active cubie's packed orientation against the starting state:

```

bool changed = false;
for i in active:
    cur = cube[ActiveCubies[i]]
    if (cur != Cubies[ActiveCubies[i]]) changed = true # did the prefix move the cluster?
    ... accumulate error ...
if (!changed) fitness += LARGE # a no-op on the active cluster is not a solution

```

This is safe: a prefix leaving the cluster untouched has fitness at least the baseline (it never improves), so the penalty discards nothing useful. Its advantage over a fitness-based check is decisive on a plateau — where the fitness is flat but the arrangement genuinely changes — because comparing fitness would falsely report "no change" exactly where the search is stuck, whereas comparing orientation sees the real move. With this penalty the per-generation best is always a real cluster-changing move; in the endgame, the least-worsening one.

3.8 Stall-driven perturbation

The plain rule of applying the best only when it improves is pure greedy descent, which stalls in local minima and produces a heavy-tailed runtime distribution. When no improvement occurs for `STALL` consecutive invocations, the host applies the best regardless — and because the dead-move penalty guarantees a real move, this is a genuine perturbation that escapes the basin (an iterated-local-search kick). The trigger is a single condition, `best.Fitness < Score OR Stall >= STALL`. Crucially the kick fires at every local minimum, not only the global endgame; since every cluster has its own coupled mini-endgame, the same mechanism completes clusters at each stage, which is what makes multi-cluster cubes such as the 3^4 tractable.

4. GPU Implementation

4.1 Compute-shader pipeline

The algorithm runs as OpenGL compute programs sharing storage buffers. Per generation the host issues: (1) evaluate the parent population, writing each best-prefix fitness (with the dead-move penalty); (2) a bitonic sort with the uniqueness penalty; (3) read back the best and stop early if it beats the current baseline; (4) selection, crossover and macro-mutation, writing children into a second buffer. The two population buffers are ping-pong: parents at binding 0, children at binding 1, roles swapped after each generation. Only the best individual is read back per generation; all heavy work stays on the device.

4.2 Packed orientation

A cubie's orientation is a signed permutation matrix — exactly one non-zero entry, +1 or -1, per row and column — packed into one 32-bit integer: N rows of `BITS_PER_ROW` bits, each holding the column index of its non-zero entry (`BITS_FOR_COL = 2` for $N \leq 4$, else 3) plus one sign bit. The identity row r encodes the value r , so a solved cubie packs to a fixed constant and a single 32-bit integer suffices up to eight dimensions. Applying a move is pure bit manipulation of two rows; the L1 distance and active-axis count are read off directly.

4.3 Buffers, single-source scoring, parallel evaluation

Compile-time constants (dimension, size, population, genes) are injected into the shader source as preprocessor defines, since they size fixed arrays; the shaders are recompiled when the cube dimension or size changes. Cluster sizes are passed as uniforms, not read from buffer lengths, since a first cluster has an empty solved-set and a zero-size

buffer length is unreliable across drivers. Two evaluation kernels are provided: a micro kernel (one thread per individual, cube in registers) for small cubes, and a macro kernel (one thread block per individual, cube in shared memory, error summed by a logarithmic parallel reduction) for larger ones.

Crucially, the acceptance baseline is computed by a single source of truth on the device: a one-thread ScoreCube kernel scores the current cube with the same scoreState function the evaluators use, and the host reads back one float. An earlier host-side mirror of the metric was removed — keeping two copies in sync across the CPU and the shader was a recurring source of silent drift in the best.Fitness < Score test. The buffer layout is summarized in Table 1.

Binding	Buffer	Contents
0	Population	Parent individuals (fitness, best-prefix length, genes)
1	NewPopulation	Children written by crossover (ping-pong)
2	Cubies	Packed starting orientation, one uint per cubie (shared)
3	FreeMoves	Legal move codes for the active cluster
4	SolvedCubies	Home indices of already-solved cubies
5	ActiveCubies	Home indices of the active cluster
6	Planes (UBO)	Axis pairs of the rotation planes
7	SeedMoves	Per-individual solving sequences (initialization / mutation)
8	ScoreBuffer	Baseline cube score written by the ScoreCube kernel

Table 1. Storage-buffer registry shared by the compute programs.

5. Results and Discussion

The solver arranges the cube cluster by cluster from the centre outward, driven only by the generated seeds, crossover, macro-mutation, the sort-embedded unique selection, and the dead-move/stall pair — with no stored macro-move database. In this configuration it solves the 6x6x6 cube in about ten minutes, and finishes clusters at least as quickly as the previous CPU solver. It also solves, for the first time, the full four-dimensional 3^4 cube (81 cubies, several cluster types) in about two minutes and roughly 12000 algorithm invocations.

The dead-move penalty and the stall perturbation are together responsible for the largest reduction of the heavy runtime tail. On the 2^4 cube — a single coupled cluster of corner cubies, the hardest single-cluster case — the mean number of algorithm invocations dropped from about 27000 to about 3800 ($\sim 7x$) and the worst case from about 136000 to about 16000 ($\sim 8.6x$), with the median falling from about 6500 to about 1400, relative to the version before this change. The other decisive, mutually reinforcing elements are the macro-mutation (without which the seeds vanish after the first generation) and the sort-embedded unique selection (which produced the qualitative jump from "stuck" to reliable cluster completion).

The fitness is now computed by one function on the device and read back for the acceptance baseline, removing the earlier host/device duplication; a bit-exact cross-check between a CPU port of the move kernel and the shader was used during development to guarantee that the packed simulation matches the reference. Because the costly per-candidate decomposition in seed generation runs only once per invocation, its cost is negligible against the evolutionary loop; where it is not, the diagonal-count metric of Section 3.6 removes the decomposition entirely at the price of two decomposition trees. The compute path scales with the population and cube size and is bounded by device memory for the packed representation, which grows only linearly in the number of cubies.

6. Conclusions

We generalized a staged, cluster-by-cluster genetic solver of the full Rubik's supercube to arbitrary dimension and executed it entirely on the GPU. The key ideas are a fitness that measures the true number of moves to solve each cubie (with an endgame reweighting that defeats the mono-twist), genetic operators that build commutators and continually re-inject cubie-solving sequences, a unique selection folded into the sort, a reject-free greedy generator of seeds covering every move-reachable decomposition tree, and — decisive for the runtime tail — a near-free dead-move penalty paired with a stall-driven perturbation that escapes local minima at every stage, all scored by a single source of truth on the device. Together they solve high-dimensional cubes within minutes without a macro-move database. Future work includes exploiting the symmetry of the hypercube to shrink the search space, learning and reusing cluster-finishing macros across cube sizes, and replacing the legacy immediate-mode display

with a retained-mode renderer.

References

- [1] R. Świta, Z. Suszyński. Solving Full $N \times N \times N$ Rubik's Supercube Using a Genetic Algorithm. *International Journal of Computer Games Technology*, vol. 2023, art. 2445335, 2023.
- [2] T. Rokicki, H. Kociemba, M. Davidson, J. Dethridge. The Diameter of the Rubik's Cube Group Is Twenty. *SIAM J. Discrete Math.*, 27(2):1082-1105, 2013.
- [3] E. Demaine, M. Demaine, S. Eisenstat, A. Lubiw, A. Winslow. Algorithms for Solving Rubik's Cubes. In *ESA 2011*, LNCS 6942, pp. 689-700, Springer, 2011.
- [4] R. E. Korf. Finding Optimal Solutions to Rubik's Cube Using Pattern Databases. In *AAAI-97*, pp. 700-705, 1997.
- [5] F. Agostinelli, S. McAleer, A. Shmakov, P. Baldi. Solving the Rubik's Cube with Deep Reinforcement Learning and Search. *Nature Machine Intelligence*, 1:356-363, 2019.
- [6] N. El-Sourani, S. Hauke, M. Borschbach. An Evolutionary Approach for Solving the Rubik's Cube Incorporating Exact Methods. In *EvoApplications 2010*, LNCS 6024, pp. 80-89, Springer, 2010.
- [7] M. Luby, A. Sinclair, D. Zuckerman. Optimal Speedup of Las Vegas Algorithms. *Information Processing Letters*, 47(4):173-180, 1993.
- [8] G. H. Golub, C. F. Van Loan. *Matrix Computations*, 3rd ed. Johns Hopkins University Press, 1996.